

Lesson:



Problems based on HashMaps



Pre Requisites:

- What is Hashing
- What are Hash Functions
- Collisions
- Load Factor
- Introduction to HashMap class
- HashMap Hierarchy
- Working with the HashMap Class

List of concepts involved :

- Problems based on Hashmaps

Problem 1: Comparison Type(Single mapping)

Check whether two Strings are anagram of each other. Return true if they are else return false.

An anagram of a string is another string that contains the same characters, only the order of characters can be different. For example, "abcd" and "dabc" are an anagram of each other.

Input1:

listen
silent

Output1:

true

Input2:

anagram
grams

Output2:

false

Explanation:

- We need to figure out the criteria of two strings to be anagrams.
The first criteria is that length of 2 strings has to be the same for them to be anagrams.
Secondly, the frequency of every character in a string has to be equal to the frequency of every character in the other string.
No character to be extra or different in any of the strings.
Thus, to maintain the count of the frequency for every character in the first string, we use a HashMap.
Now, we can follow 2 approaches here: maintain a HashMap for both the strings and in the end just compare both the HashMaps, if they are equal we return true else false.
This can be optimized by using only 1 HashMap. We store frequency for each character in any one of the strings. Then traverse over the other string, and for every character, first check if it is present in the HashMap or not. If it is not present, the strings cannot be anagrams anyways, so we return false. Else, we decrease its current frequency from the HashMap.
- Secondly, let's consider if we can design a hash function which will give the same value for two strings if they are anagram of each other. In anagrams, only frequency matters, thus we can sort the strings through the hash function, i.e. $\text{hash}(\text{str}) = \text{sort}(\text{str})$. Now if $\text{hash}(\text{str1}) = \text{hash}(\text{str2})$ means str1 and str2 are anagrams.

Code:

<https://pastebin.com/CYBzyjA4>

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Content
Enter the two strings:
listen
silent
true

Process finished with exit code 0
```

Problem 2: Dual Mapping Type

Check whether two Strings are isomorphic of each other. Return true if they are else return false.

Two strings are isomorphic of each other if there is a one-to-one mapping possible for every character of the first string to every character of second string and all occurrences of every character in first string maps to the same character in the second string.

Input1:

aab

xyy

Output1:

true

Input2:

abcdec

Output2:

viouog

Explanation:

- Two strings are considered to be isomorphic of each other, if there is a one to one mapping between all characters of the first to the second string.

- Consider three cases:

aabb and ccdd

This is an example of isomorphic strings because both the a's in the first string are mapped to the same character, i.e. c in the second string. Also for b, both are mapped to d.

aabb and ccde

This is not an example of isomorphic strings because both the a's in the first string are mapped to the same character, i.e. c in the second string but for b, one of them is mapped to d whereas the other is mapped to e.

aabb and cccd

This is not an example of isomorphic strings because both the a's in the first string are mapped to the same character, i.e. c in the second string but for b, one of them is mapped to c which has already been mapped to a from the first string.

- To maintain one-to-one mapping, we will use a HashMap here, where all characters from the first string will be the keys mapped with the corresponding character in the second string. The characters of the second string will be the values.

- If the length of both the strings is unequal, they cannot be isomorphic.
- We will traverse over the length of the strings, for every character in the first string, check if it is already present in the hashmap or not. If already present, check if the value of this key is the same as the current character of the second string. If it is same, the match is correct and move to the next index, otherwise we have found a mismatch, and the strings are not isomorphic. Else, put the mapping to the hashmap.

Code:

<https://pastebin.com/8GfinLMq>

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Content
Enter the two strings:
aab
xxy
true

Process finished with exit code 0
```

Problem 3: Pair sum

Given an array of length n and a target, return a pair whose sum is equal to the target. If there is no pair present, return -1 .

Input1:

$n = 4$
 $arr[] = \{1, 5, 7, -1\}$
 $target = 6$

Output1:

1 5

Input2:

$n = 10$
 $arr[] = \{10, 12, 10, 15, -1, 7, 6, 4, 2, 1\}$
 $target = 11$

Output2:

10 1

Explanation:

- To find a pair and compare its sum to the target, we need to consider every element from the start and look for an element that completes its pair whose sum equals to the target.
- Instead of using nested loops, we can reduce the time complexity by using a HashMap here.
- First store all the elements in the HashMap, if there is any duplicate, we increment its value. So, we are storing element to its frequency as a pair.
- Traverse over the array and for every element, check if there is target-element present in the HashMap. If yes, it means we have found a pair and return the pair.
- If after traversing over the entire array, we find no such pair, we return -1 .

Code: <https://pastebin.com/WDLLR9xK>

```

/Library/Java/JavaVirtualMachines/jdk-19.jdk/Content
Enter the length of the array:
4
Enter the elements of the array:
1 5 7 -1
Enter the target:
6
1 5

Process finished with exit code 0

```

Problem 4:

Given an array `arr[]` of length `N`, find the length of the longest subarray with a sum equal to 0.

Input1:

`n = 8`
`arr[] = {15, -2, 2, -8, 1, 7, 10, 23}`

Output1:

5

Explanation: The longest subarray with elements summing up to 0 is `{-2, 2, -8, 1, 7}`

Input2:

`n = 3`
`arr[] = {1, 2, 3}`

Output2:

0

Explanation:

- Let's say there exist a subarray `a[L...R]` which has sum 0 thus
 $a[L] + a[L+1] + \dots + a[R-1] + a[R] = 0$ or $(a[0] + a[1] + \dots + a[R]) - (a[0] + a[1] + \dots + a[L-1]) = 0$
i.e $\text{prefix_sum}(R) - \text{prefix_sum}(L-1) = 0$ or $\text{prefix_sum}(R) = \text{prefix_sum}(L-1)$.
- Now, if we take the prefix sum of the array then our question reduces to the maximum distance between two elements which have same value.
- Let's try to solve our problem using a HashMap.
- The algorithm is the same, we will keep a track of the prefix sum for every index in the HashMap. At every index, we basically need to check if there was any index prior that had the same prefix sum, and if it did we must have stored it in the map. This is where we will check the hashmap if it contains the same prefix sum as that of the current index, if so, we have a subarray and just check if it is the longest yet by subtracting the indices. If not, we will store it in the map, to check for the further indices.
- Create a variable (`sum`), length (`max_len`), and a hash map (`map`) to store the sum-index pair as a key-value pair.
- HashMap will store Integer, Integer pairs.
- Traverse the input array and for every index, update the value of `sum = sum + array[i]`.
- Check every index, if the current sum is present in the hash map or not.

- If present, update the value of max_len to a maximum difference of two indices (current index and index in the hash-map) and max_len.
- Else, put the value (sum) in the hash map, with the index as a key-value pair.
- Print the maximum length (max_len).

Code:

<https://pastebin.com/NpWzYx3k>

```
/Library/Java/JavaVirtualMachines/jdk-  
Enter the length of the array:  
8  
Enter the elements of the array:  
15 -2 2 -8 1 7 10 23  
5  
  
Process finished with exit code 0
```

Upcoming Class Teasers:

- HashSets